

AI ACADEMY, NATIONAL AI CENTER

AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Final Project Brief

AI Engineering: Software Engineering Final Project

Released:	May 13, 2026	Due:	May 23, 2026 at 23:59 (UTC+4)
Team size:	3 students (2 or 4 with approval)	Workload:	~30–33 person-hours around 1 week
Version:	v2.0	Submission:	GitHub repo + Moodle submissions

At a glance

What this project is. The Software Engineering section taught you Python end-to-end: control flow, data structures, OOP, file I/O and APIs, web scraping, concurrency (threading, multiprocessing, asyncio), code quality, testing, debugging, and Docker. This final project is where you put all of it together into one shippable codebase.

What it is NOT. You are not expected to train, fine-tune, or evaluate any machine-learning model. You have not yet covered ML in this academy. Every topic uses pre-trained models exposed through external APIs (Claude, GPT-4o, Gemini, embedding endpoints, etc.). *The AI module is provided.* Your job is to wrap, orchestrate, persist, parallelize, validate, test, containerize, and ship it.

Three deliverables, one deadline. On **May 23, 2026 at 23:59 (UTC+4)** you submit one ZIP archive containing the following:

1. Source **code** (GitHub tag `v1.0-final`)
2. Compiled **report** (`report/report.pdf`, from the provided LaTeX template)
3. **Presentation deck** (~10 slides, from the provided Beamer template) for the oral defense

Contents

1	What You Receive and What You Build	2
1.1	What you will build (every topic)	3
2	Course Context and Learning Outcomes	3
3	Project Format	4
3.1	Teams	4
3.2	Topic selection	4
3.3	Two-week timeline	4
4	Common Software Engineering Requirements	4
4.1	Repository and configuration	4
4.2	Object-oriented design	5
4.3	Wrapping the provided AI module	5
4.4	Concurrency	5
4.5	Robustness and error handling	6
4.6	Testing	6
4.7	Deployment	6

4.8	What you may NOT do	7
5	Topic Specifications	7
5.1	Topic 1 – Smart Lost & Found	7
5.1.1	What we provide in <code>topic-1-lost-and-found/</code>	7
5.1.2	What you must build	7
5.1.3	Minimum runnable demo we will grade	8
5.2	Topic 2 – AI Food Analyzer	8
5.2.1	What we provide in <code>topic-2-food-analyzer/</code>	8
5.2.2	What you must build	8
5.2.3	Minimum runnable demo we will grade	8
5.3	Topic 3 – AI News Briefing Service	9
5.3.1	What we provide in <code>topic-3-news-briefing/</code>	9
5.3.2	What you must build	9
5.3.3	Minimum runnable demo we will grade	9
5.4	Topic 4 – Async Research Assistant	9
5.4.1	What we provide in <code>topic-4-research-assistant/</code>	9
5.4.2	What you must build	10
5.4.3	Minimum runnable demo we will grade	10
6	The AI Module — Provider-Agnostic Design	10
7	Deliverables	11
8	Grading Rubric	11
8.1	Automatic deductions	12
9	Advanced Bonuses (Optional)	13
10	Submission Instructions	13
11	Academic Integrity and Allowed Tools	14
12	Self-Assessment Checklist	14
A	Reference Project Layout	15
B	Recommended Libraries	16
C	Working with the Provided AI Package — Quick Reference	16
D	Common Pitfalls	16

1. What You Receive and What You Build

For each of the four topics, the course provides a folder `topic-N-<name>/` containing:

- A runnable, provider-agnostic `ai/` Python package that talks to the AI APIs.
- Concrete provider adapters for **Anthropic**, **OpenAI**, and **Google Gemini** – pick whichever your team has API access to via an environment variable.
- Pydantic schemas for AI inputs and outputs.
- Sample input data (a handful of images, RSS feeds, or example queries).

- A `demo_ai.py` script that exercises the AI module end-to-end on the sample data.
- A small `tests/` folder with offline (mocked) smoke tests for the AI layer. **These tests are a contract you must not break.**
- A topic-specific `TOPIC.md` explaining what the AI module does and its inputs/outputs.

Read this twice

The provided AI package is intentionally minimal and well-typed. It does *not* include: configuration management, persistence, retries, rate limiting, logging, batching, concurrency, a CLI or HTTP server, input validation beyond basic type checks, or production-grade error handling. **Building all of those around the AI module is the assignment.**

1.1 What you will build (every topic)

Around the provided AI package, your team must implement:

1. A typed **configuration** layer reading from `.env`.
2. **Persistent storage** (PostgreSQL or AsyncPG minimum; filesystem for blobs).
3. A **concurrency** layer that parallelizes the I/O-bound work appropriate to the topic.
4. A **retry and rate-limit** policy wrapping every external call (including the provided AI calls).
5. **Input validation** at every entry point (CLI, HTTP, file, env).
6. Structured **logging** via the `logging` module.
7. A **CLI** (mandatory) and an **HTTP API** (recommended; required for Topic 1 and Topic 2).
8. A **test suite** with $\geq 60\%$ coverage and zero live-network dependencies.
9. A **Dockerfile** that builds and runs the demo end-to-end with one command.
10. A complete `README.md`, the **report** in `report/report.pdf`, and the **presentation deck**.

2. Course Context and Learning Outcomes

This is a group final project closing the Software Engineering portion of the AI Engineering track. By the end of it you should be able to demonstrate:

- **Architecture:** decompose a real product into modules, classes, and clear interfaces; treat the provided AI package as one well-defined external dependency.
- **Concurrency:** pick the right tool for the job (asyncio for I/O-bound, multiprocessing for CPU-bound, threading where appropriate) and justify the choice with a benchmark.
- **API integration:** consume third-party REST APIs robustly, with retries, timeouts, rate-limit handling, and structured error reporting.
- **Robustness:** validate inputs, handle exceptions intentionally, fail gracefully, and log meaningfully.
- **Testing:** write unit and integration tests with pytest, mock external dependencies, and reach a stated coverage target.
- **Reproducibility:** package the project with a pinned `requirements.txt`, a Dockerfile, and a `README.md` with set-up and run commands.

- **Collaboration:** use Git/GitHub with branches, pull requests, and reviews; distribute work fairly across the team.
- **Communication:** write a technical report and present a 10-minute oral defense.

3. Project Format

3.1 Teams

Teams of 3 students. Teams of 2 or 4 are allowed only with the instructor’s prior written approval. Each team picks exactly one of the four topics in §5. One member from each team should send their group composition (Name Surname Group format) and their project topic to the email address sultan.musayeva@aiacademy.az

3.2 Topic selection

The four topics were selected from a class-wide questionnaire. You may pick any of them; multiple teams may pick the same topic, but their codebases must be developed independently. Identical or near-identical code shared between teams will be reported to the academy administration as plagiarism.

3.3 Two-week timeline

The recommended milestone schedule below assumes a 12-day window from release to deadline. Adjust to your team’s availability, but **do not start coding the SE layer in week 2**.

Day	Phase	Goal
Day 1–2	Setup	Pick topic; create repo; everyone runs <code>demo_ai.py -offline</code> and the provided smoke tests; agree on architecture diagram and per-person task split; open a tracking issue with the 10 deliverables as checkboxes.
Day 3–5	Skeleton	<code>config.py</code> , storage, and one CLI command working end-to-end against the provided AI module. First PR merged. First own tests written.
Day 6–8	Concurrency	Parallel pipeline working. Sequential-vs-concurrent benchmark captured in the README (numbers and command lines). HTTP API stood up (Topics 1, 2).
Day 9–10	Robustness	Retries, rate limit, validation, structured logging. Inject failures and verify graceful degradation. Coverage reaches 60%.
Day 11	Containerise	Dockerfile builds; <code>docker run</code> demonstrates the full demo. Pin all versions. Smoke tests still pass.
Day 12	Polish & submit	Report drafted from template; slides prepared; contribution statement signed; tag <code>v1.0-final</code> ; submission to Moodle done before May 23, 2026 at 23:59 (UTC+4).

Reality check. The most common failure mode is leaving robustness, tests, and Docker for the last day. Robustness work *always* reveals architectural problems that take time to fix cleanly. Front-load it.

4. Common Software Engineering Requirements

These apply to all four topics. Topic-specific additions are in §5.

4.1 Repository and configuration

- GitHub repository, public or shared with the instructor.

- Branch-based workflow: `main` is protected; features land via pull requests with at least one teammate's review.
- Every team member must have a non-trivial commit history. Target: no team member below ~20% of total commits. Severe imbalance (<10%) without a documented reason incurs a deduction.
- `README.md` with: project description, setup steps, environment variables, how to run, how to test, sample command lines, and a Docker quick-start.
- `.env.example` listing all expected environment variables. Do not commit real keys.
- `.gitignore` excludes `.env`, `__pycache__`/, `*.pyc`, virtual environments, and any local databases or downloaded data.
- A single `config.py` (we recommend `pydantic-settings` or `dataclasses`) reads environment variables and exposes typed settings to the rest of the code. **Hard-coded secrets anywhere in the codebase are an automatic fail of the Robustness criterion plus a fixed deduction.**

4.2 Object-oriented design

- Use classes for stateful components: API clients, storage repositories, the pipeline orchestrator, the rate-limiter. Use plain functions for pure transformations.
- Define data models with `pydantic` models. The provided AI package already exposes its inputs/outputs as `pydantic` models. **No naked dictionaries flowing across module boundaries.**
- At least one example of inheritance or composition that you can defend in the report. The provided AI package contains a clean example: an abstract `LLMProvider` with concrete subclasses – study and reference it, but show your own example in your code.
- Type hints on every public function and method. Run `mypy` or `pyright` at least once and report results.

4.3 Wrapping the provided AI module

The AI package is a clean, but thin, library. It does not implement retries, caching, rate limiting, or logging – those are your responsibility, and how well you wrap the AI module is a major part of your grade.

- Wrap every call to `ai.*` in a retry decorator with exponential backoff for transient errors (network errors, 5xx, 429).
- Add a timeout to every call.
- Log inputs (without secrets), outputs, and timings at `INFO`; full payloads at `DEBUG`.
- Cache deterministic AI outputs where the topic allows it (e.g. embeddings of identical text).
- Validate the structure of the model's response before passing it downstream. The provided code already validates a JSON schema; you must still defend against semantically wrong outputs.
- Bound parallelism with a semaphore to respect provider rate limits.

4.4 Concurrency

At least one of the following must be genuinely used and benchmarked against a sequential baseline:

- `asyncio + aiohttp` (or `httpx`) for I/O-bound parallelism.
- `concurrent.futures.ProcessPoolExecutor` or `multiprocessing` for CPU-bound work.
- `concurrent.futures.ThreadPoolExecutor` where it is the right tool (e.g. wrapping a blocking SDK).

Your report must contain a wall-clock comparison (N requests sequential vs. concurrent) for at least one realistic workload, on the same machine, with caches cleared.

4.5 Robustness and error handling

- Every external call (HTTP, file, DB, AI module) is wrapped in explicit error handling. `except Exception: pass` is forbidden. Bare `except:` is forbidden.
- Implement retries with exponential backoff for transient HTTP failures. `tenacity` is recommended.
- Honour API rate limits. At minimum, a token-bucket or simple sleep-on-429 strategy. Document the bucket size in your report.
- Validate user input. Reject malformed payloads with a clear error message, not a stack trace.
- Use the `logging` module (not `print`) for runtime diagnostics; configurable log level via env var.
- **Graceful degradation:** if one upstream source fails, the pipeline should still produce a useful result with a clear note about what was missing.

4.6 Testing

- Use `pytest`. Tests live in `tests/`.
- All tests must run offline. Mock the AI module and any external HTTP using `unittest.mock`, `pytest-httpx`, `respx`, or `responses`. Tests that hit the network in CI are an automatic deduction.
- Cover at minimum: one happy-path end-to-end test, three unit tests per core module, three error-path tests, and one concurrency test (e.g. that `asyncio.gather` behaves as expected when one task raises).
- Target coverage: $\geq 60\%$ measured by `pytest-cov`. Print the report in your README or in CI.
- **The smoke tests we shipped in the AI folder must continue to pass after your integration. Do not delete or weaken them.**

4.7 Deployment

- A working Dockerfile that builds an image containing your application.
- The image must run end-to-end with one command (`docker run ...`) given the right environment variables.
- Pinned versions in `requirements.txt` (or `pyproject.toml`). **No `pip install <package>` without a version.**
- For Topics 1 and 2 (HTTP server required), expose the server port via `EXPOSE` and document the `curl` call in the README.

4.8 What you may NOT do

- Do not modify the AI module's public interface. You may add wrappers around it; you may not edit `ai/` files.
- Do not train or fine-tune any ML model. You haven't covered the maths or the workflow yet.
- Do not paste code wholesale from another team or from a tutorial without attribution. AI-assisted coding is allowed but must be disclosed (§11).
- Do not commit real API keys or scraped data containing personally identifiable information. Sample data only.

5. Topic Specifications

Each topic below summarises: (i) the problem, (ii) what is in the provided AI folder, (iii) what you must build around it, and (iv) the minimum runnable demo we will grade against. **Pick exactly one.**

5.1 Topic 1 – Smart Lost & Found

Problem. Users register lost items (image + short description) and found items (image + short description). The system returns the most likely matches between the two pools. The match is computed from a vision-language description plus an embedding-based similarity score over the descriptions.

5.1.1 What we provide in `topic-1-lost-and-found/`

- `ai/vlm.py – describe_item(image_path, user_text)` returns a structured `ItemDescription` (object class, colour(s), brand, distinguishing marks, location hints, confidence).
- `ai/embedding.py – embed(text)` returns a unit-normalized vector from the chosen embedding provider.
- `ai/similarity.py – cosine(a, b)` and `top_k(query, candidates, k)` pure-NumPy helpers.
- `ai/schemas.py` – the `ItemDescription` pydantic schema.
- `data/` – 12 sample images split into `lost/` and `found/`.
- `demo_ai.py` – runs the full AI pipeline on the samples and prints top-3 matches.

5.1.2 What you must build

- **HTTP API** (FastAPI or Flask): `POST /items/lost`, `POST /items/found`, `GET /items/{id}/matches?k=N`, `GET /items?status=...`
- **CLI** mirroring the same operations: `register-lost`, `register-found`, `search-matches`, `list`.
- **Persistent storage:** PostgreSQL or AsyncPG for metadata (item id, status, user-supplied text, VLM description, embedding bytes, timestamp); filesystem for image blobs under a configurable directory.
- **Concurrency:** when an item is registered, the embedding extraction and similarity scoring against the opposite pool run concurrently for batched registration. A single registration may be synchronous; a batch must use `asyncio.gather` (or a worker pool).

- **Robustness:** validate image MIME type (JPEG/PNG only), enforce a configurable size limit (default 5 MB), reject corrupted files with a clean error.
- **Caching:** re-embedding the same description twice within a session must hit a cache.

5.1.3 Minimum runnable demo we will grade

A scripted scenario (`scripts/demo.py`) that registers the lost and found items in `data/`, then queries one specific lost item and prints the top-3 matches with similarity scores and the VLM's reason for the match. The same scenario also runs against the HTTP server via `curl` commands documented in the README.

5.2 Topic 2 – AI Food Analyzer

Problem. The user uploads a photo of a meal. The system identifies the ingredients with estimated portion sizes, retrieves nutritional data per ingredient, and returns total calories plus a macronutrient breakdown.

5.2.1 What we provide in `topic-2-food-analyzer/`

- `ai/vlm.py` – `identify_ingredients(image_path) -> list[Ingredient]` where each `Ingredient` has `{name, estimated_grams, confidence}`. JSON-schema validated.
- `ai/nutrition.py` – a pluggable `NutritionProvider` interface with one concrete adapter for the USDA FoodData Central API (free; key from `api.data.gov`).
- `ai/calculator.py` – pure function `compute_totals(ingredients, facts) -> Nutrition` (kcal, protein, carbs, fat).
- `ai/schemas.py` – the `Ingredient`, `NutritionFacts`, and `Nutrition` pydantic schemas.
- `data/` – 16 meal photos.
- `demo_ai.py` – analyses one sample photo and prints the totals.

5.2.2 What you must build

- **CLI and HTTP server.** CLI: `python -m foodanalyzer analyze <path>`. HTTP: POST `/analyze` accepting a multipart image upload, returning structured JSON.
- **History log:** every analyzed meal is stored in PostgreSQL or AsyncPG (timestamp, image path, ingredients, totals).
- **Concurrency:** when the VLM returns N ingredients, the nutrition lookups for all N run in parallel via `asyncio.gather`. A single ingredient lookup is fine sequentially.
- **Caching:** nutrition lookups for the same ingredient string within a configurable TTL (default 24h) must hit a local cache.
- **Robustness:** graceful behaviour when the VLM cannot identify the meal (return a structured “unknown” result, not a crash); handle USDA outages with a clear error path.

5.2.3 Minimum runnable demo we will grade

`python -m foodanalyzer analyze data/sample_meal.jpg` prints a table of ingredients, their grams, kcal, protein/carbs/fat columns, and a totals row. The same call works through `curl` against the running server.

5.3 Topic 3 – AI News Briefing Service

Problem. A scheduled service that fetches articles from multiple news sources concurrently, deduplicates near-identical stories, categorizes them by topic, and produces a personalized daily digest for a user given their interests.

5.3.1 What we provide in `topic-3-news-briefing/`

- `ai/llm.py` – `summarize_and_label(article)` -> `LabeledSummary`, returning `{summary, topic, sentiment}`; topic is one of a fixed enumeration.
- `ai/embedding.py` – the same provider-agnostic `embed` as Topic 1.
- `ai/dedup.py` – pure helpers: `url_canonicalize(url)`, `content_hash(text)`, and a shingle-based near-duplicate checker (`near_duplicate(a, b, threshold)`).
- `ai/schemas.py` – `Article`, `LabeledSummary`, `Digest`.
- `data/` – 5 RSS feed URLs, 2 sample HTML pages, sample user profile.
- `demo_ai.py` – pulls articles from the HTML samples, summarises a few, and prints a Markdown digest.

5.3.2 What you must build

- **Concurrent ingestion** of at least 5 sources using `asyncio` + `aiohttp`. At least one source must be direct-scraped HTML.
- **Two-stage deduplication** wired around the provided helpers: cheap first, semantic second. Document your thresholds.
- **User profile** stored in PostgreSQL or AsyncPG or JSON: preferred topics + sources to exclude.
- **Output:** a Markdown digest written to `digests/YYYY-MM-DD-<user>.md` with sectioned summaries and source links.
- **Scheduling:** a CLI command `run-daily` that performs the full pipeline. Bonus if you wire it through APScheduler or a Docker entrypoint loop.

5.3.3 Minimum runnable demo we will grade

`python -m newsbrief run-daily -user khagani` produces a Markdown digest with 5–10 deduplicated, summarized, topic-tagged articles relevant to the user’s interests, sourced from at least 5 providers. The wall-clock time of the parallel ingestion vs. sequential is reported in the README.

5.4 Topic 4 – Async Research Assistant

Problem. The user asks a research question. The system queries Wikipedia, arXiv, and a web-search API in parallel, retrieves relevant excerpts, then synthesises a single answer with inline citations.

5.4.1 What we provide in `topic-4-research-assistant/`

- `ai/sources.py` – async fetchers `fetch_wikipedia(query)`, `fetch_arxiv(query)`, `fetch_web(query)`. The web-search adapter supports Tavily, Serper, and `duckduckgo-search`, selected by `env var`.

- `ai/synthesizer.py` – `synthesize(question, sources) -> AnswerWithCitations` prompting the chosen LLM to produce an answer with bracketed source indices.
- `ai/schemas.py` – `Source`, `AnswerWithCitations`, `Citation`.
- `data/` – 5 example research questions of varying difficulty.
- `demo_ai.py` – answers two example questions and prints the cited results.

5.4.2 What you must build

- **Concurrent orchestration:** all three sources queried via `asyncio.gather` with **per-source timeouts** and **graceful degradation** (if arXiv fails, the answer is still produced from the other two).
- **Caching:** keyed by (source, query) pair with a configurable TTL.
- **CLI:** `python -m researcher ask "your question"` prints the synthesized answer and the references; `-no-cache` bypasses the cache.
- **Citation tracking:** each fact in the final answer carries a numeric reference; references list title + URL.
- **Per-source fallback:** a CLI flag `-sources wiki,arxiv` restricts to a subset.

5.4.3 Minimum runnable demo we will grade

A scripted run of all 5 example questions in `data/` producing one cited answer per question, plus a wall-clock comparison of parallel vs. sequential fetching written in the README.

6. The AI Module — Provider-Agnostic Design

The provided `ai/` package centralizes provider selection through a single environment variable.

```
# In .env (you create this):
LLM_PROVIDER=anthropic          # or "openai" or "gemini"
LLM_MODEL=claude-sonnet-4-6     # provider-specific model id
ANTHROPIC_API_KEY=sk-ant-...

EMBEDDING_PROVIDER=openai       # or "gemini"
EMBEDDING_MODEL=text-embedding-3-small
OPENAI_API_KEY=sk-...
```

The AI package internally uses the abstract-base-class pattern:

```
1 class LLMProvider(ABC):
2     @abstractmethod
3     def complete(self, prompt: str, *, json_schema: dict | None = None) -> str: ...
4
5 class AnthropicProvider(LLMProvider): ...
6 class OpenAIProvider(LLMProvider): ...
7 class GeminiProvider(LLMProvider): ...
8
9 def get_llm() -> LLMProvider:
10     """Factory: returns the configured provider based on env."""
```

You should study this pattern; your own code should follow the same shape (e.g. for storage backends, news sources, cache backends).

7. Deliverables

A team submission consists of **four items**, all due by **May 23, 2026 at 23:59 (UTC+4)**:

1. **GitHub repository** link (put it in the report PDF) containing your code, tests, the Dockerfile, the README, and `.env.example`. The provided `ai/` package is included unmodified.
2. **Compiled report** (`report/report.pdf`) following the provided LaTeX template (`templates/REPORT_TEMPLATE.tex`) 8–10 pages.
3. **Presentation deck** (~10 slides, ~10 minutes) following the provided Beamer template (`templates/SLIDES_TEMPLATE.tex`). Used in the oral defense the week after submission.
4. **Contribution statement** (one page, signed by all members): which modules, which PRs, which tests each member owned. Follow `templates/CONTRIBUTION_STATEMENT.md`.

Sample run artefacts. For the chosen topic, include the output of one full run (digest `.md`, JSON answer, screenshots, CSV — whatever your topic produces) under `artefacts/` in the repo.

Oral defense. A 10-minute presentation + Q&A is held in the week following submission. We will randomly select files for walk-through during Q&A.

8. Grading Rubric

The total is 100 points, split between Code (60%), Report (25%), and Presentation (15%).

Criterion	Weight	What we check
Code – 60%		
Correctness & Functionality	22%	End-to-end demo works; minimum runnable demo of the chosen topic produces sensible output; provided AI smoke tests still pass.
Architecture & OOP Design	13%	Clear module boundaries, dataclasses/schemas across boundaries, defensible inheritance/composition, the AI module is treated as a clean boundary (not interleaved with business logic).
Concurrency & Performance	10%	Right tool for the job; bounded parallelism (semaphore) respecting provider rate limits; sequential-vs-concurrent benchmark in the report.
Robustness & Error Handling	8%	Retries with exponential backoff, timeouts, input validation, graceful degradation, structured logging.
Testing & Code Quality	7%	No bare except . Hard-coded keys = 0 in this category. $\geq 60\%$ pytest coverage with offline tests; mocked AI module; type hints; clean style; meaningful docstrings; no dead code or TODOs in final submission. Provided smoke tests pass.
Report – 25%		
Conceptual Understanding	10%	Why these modules, why this concurrency model, why this storage, why this provider, how the AI module is wrapped, what the trade-offs are.
Analysis & Interpretation	9%	Sequential-vs-concurrent timings with the new bottleneck named; failure-mode analysis with at least one concrete incident; limitations honestly reported.
Clarity & Professionalism	6%	Architecture diagram, well-formatted code blocks, no leftover guidance text, consistent language, references where appropriate.
Presentation – 15%		
Technical Depth	8%	The team can answer architectural and concurrency questions on the spot. Random-file walk-through demonstrates ownership of the code.
Clarity & Time Management	4%	10 minutes used well; slides are readable and uncluttered; diagrams over text where possible.
Team Distribution	3%	All three members speak; each demonstrates knowledge of components they contributed to.

8.1 Automatic deductions

- Hard-coded API keys anywhere in the repository: **–10 pts.**
- Tests that require live network: –5 pts.
- Missing Dockerfile or non-buildable Docker image: –5 pts.
- Severe commit imbalance ($< 10\%$ for any team member without a documented reason): –5 pts.
- TODO / placeholder comments left in final submission: –3 pts.
- Modifying the provided `ai/` package's public interface: –5 pts.
- Provided AI smoke tests broken in final submission: –5 pts.

9. Advanced Bonuses (Optional)

Strong teams who finish the core requirements ahead of schedule may pursue any of the following for up to **+10 bonus points** (capped). Each bonus must be clearly documented in the report and demonstrably tested.

Bonus	Points	What we look for
Multi-provider failover	+3	Primary LLM/embedding/web-search provider fails $\Rightarrow$ wrapper transparently retries against a configured secondary. Documented via a chaos-test that kills the primary.
Cost telemetry	+2	Every AI call records token counts and a \$-per-call estimate using a known pricing table. A CLI command cost-report prints totals for the last N hours.
GitHub Actions CI	+2	On every PR: lint (ruff or flake8), type-check (mypy or pyright), test (pytest + coverage threshold gate), build Docker image. Failing checks block merge.
OpenTelemetry tracing	+2	Every external call is a span with attributes (provider, model, latency, status). Tracing exported to console or to a docker-compose-attached Jaeger.
Web UI	+2	A simple Streamlit, Gradio, or htmx front-end exercising the same business logic the CLI / HTTP API use. (Topics 1, 2, 4.)
Streaming responses	+2	For the LLM-bound steps (Topics 3, 4 synthesis), use streaming where the provider supports it. CLI prints tokens as they arrive.
Token-aware rate limiter	+2	Beyond request-count limiting, track tokens-per-minute against the provider's documented limit and back off when nearing it.
Multi-stage Dockerfile	+1	Build stage compiles dependencies, runtime stage ships only what's needed. Final image ≤ 250 MB.

Bonuses are not a shortcut around weak fundamentals. If your core implementation is incomplete or fragile, bonus features will be ignored. We grade depth before breadth.

10. Submission Instructions

1. Tag the final commit on **main** as **v1.0-final**.
2. Submit to Moodle submission link by **May 23, 2026 at 23:59 (UTC+4)** containing:
 - GitHub repository URL and the tag name (**v1.0-final**).
 - The compiled **report.pdf**.
 - The compiled slide deck (PDF).
 - The signed contribution statement (PDF).
 - The folder of sample artefacts (or a link to it in the repo).
3. Do not push commits to **main** after the deadline. Any post-deadline commits to **main** are ignored, but their presence may incur the late-submission penalty.

Late policy. Late submissions: -10 pts per 24 hours, up to 3 days. No submissions accepted after 72h late without a documented emergency.

11. Academic Integrity and Allowed Tools

You may use AI coding assistants (Claude, ChatGPT, Cursor, Copilot, etc.) as you would in industry: **as collaborators, not as ghostwriters**. Two rules:

1. You must **understand and be able to defend every line in your repository**. The oral defense will involve walk-throughs of randomly selected files. “The AI wrote it” is not an acceptable answer.
2. **Disclose substantial AI assistance** in the report’s “Tools & Acknowledgements” section. You don’t have to log every autocomplete; you do have to acknowledge if a whole module was AI-drafted.

Identical or near-identical code shared between teams is treated as plagiarism and reported to the academy administration. Each team’s repository is checked against the others.

API keys are personal

Do not commit them, do not paste them in screenshots, do not share them with other teams. Each team uses its own keys; if you do not have access to a paid API tier, the free tiers cited in each topic’s TOPIC.md are sufficient for grading.

12. Self-Assessment Checklist

Before tagging `v1.0-final`, your team should be able to tick every box below.

Repo health

- ☐ `main` is protected; every merged PR has at least one teammate review.
- ☐ Every team member has $\geq 20\%$ of the commits.
- ☐ No `.env`, secrets, or local DB files are tracked in git.
- ☐ `.env.example` lists every variable the app reads.

Code

- ☐ `python demo_ai.py -offline` still works in every topic folder.
- ☐ `pytest tests/test_ai_smoke.py` passes (the provided smoke tests).
- ☐ `pytest -cov` reports $\geq 60\%$.
- ☐ `mypy` (or `pyright`) was run at least once; the result is in the report.
- ☐ No `except Exception: pass`; no bare `except:`; no `print` for runtime diagnostics.
- ☐ No `TODO`, `FIXME`, or placeholder comments in shipped code.

Concurrency

- ☐ A sequential-vs-concurrent benchmark is in the README with raw numbers and the command to reproduce.
- ☐ Parallelism is bounded by a semaphore; the bound is configurable.

Docker

- ☐ `docker build .` succeeds from a clean clone.
- ☐ `docker run` runs the demo end-to-end given env vars.
- ☐ `requirements.txt` has *every* dep pinned to a version.

Deliverables

- ☐ `report/report.pdf` renders from `templates/REPORT_TEMPLATE.tex`.
- ☐ Slide deck PDF renders from `templates/SLIDES_TEMPLATE.tex`.
- ☐ Contribution statement is signed by all three members.
- ☐ `v1.0-final` tag pushed.
- ☐ Submission to Moodle completed before May 23, 2026 at 23:59 (UTC+4).

A. Reference Project Layout

```

team-<name>/
|-- README.md
|-- requirements.txt
|-- pyproject.toml           # optional but encouraged
|-- Dockerfile
|-- .env.example
|-- .gitignore
|-- ai/                      # PROVIDED -- DO NOT MODIFY public
    interface
|   |-- __init__.py
|   |-- ...
|-- src/
|   |-- __init__.py
|   |-- config.py           # env -> typed settings
|   |-- models.py           # YOUR pydantic schemas for the SE layer
|   |-- services/           # wrappers around ai/, retries, logging
|   |   |-- ai_service.py
|   |   |-- ...
|   |-- core/               # business logic
|   |-- concurrency/        # async orchestration, worker pools
|   |   '-- pipeline.py
|   |-- storage/            # Your database of choice / filesystem
    persistence
|   |-- repository.py
|   |-- cli.py              # CLI entry points
|   '-- api.py              # HTTP server (Topics 1, 2)
-- tests/
|   |-- __init__.py
|   |-- conftest.py
|   |-- test_services.py
|   |-- test_core.py
|   |-- test_concurrency.py
|   '-- test_end_to_end.py
-- data/                    # sample inputs (from provided folder)
-- artefacts/               # outputs of demo runs
-- docs/
|   '-- architecture.md
'-- report/
    |-- report.tex          # from templates/REPORT_TEMPLATE.tex
    '-- report.pdf

```


B. Recommended Libraries

Concern	Recommended (pick one)
HTTP (sync)	<code>requests</code>
HTTP (async)	<code>aiohttp</code> , <code>httpx[http2]</code>
HTML scraping	<code>beautifulsoup4</code> , <code>selectolax</code>
RSS	<code>feedparser</code>
Retries	<code>tenacity</code>
Configuration	<code>pydantic-settings</code> , <code>python-dotenv</code>
Storage	<code>psychopg2</code> or <code>ayncpg (stdlib)</code> , <code>sqlalchemy</code> (optional)
HTTP server	<code>fastapi</code> + <code>uvicorn</code> , or <code>flask</code>
Embedding store	<code>NumPy</code> + cosine, <code>faiss-cpu</code> (optional)
Dedup hashing	<code>datasketch</code> (MinHash, LSH)
Wikipedia / arXiv	<code>wikipedia-api</code> , <code>arxiv</code>
Web search	<code>Tavily</code> , <code>Serper</code> , <code>Brave</code> , <code>duckduckgo-search</code>
Testing	<code>pytest</code> , <code>pytest-cov</code> , <code>pytest-asyncio</code> , <code>respx</code> , <code>responses</code>
Type checking	<code>mypy</code> or <code>pyright</code>
Logging	<code>logging</code> (stdlib) – please don't add a new framework

C. Working with the Provided AI Package — Quick Reference

Do.

- Import from `ai` and treat it like any external SDK.
- Wrap calls in your own service layer with retries, timeouts, logging.
- Cache results where it makes sense (embeddings, deterministic queries).
- Read `ai/` source code – it is short and well-commented.

Do not.

- Edit `ai/*.py` files. If you find a bug, file an issue with the instructor.
- Bypass the AI module by calling provider SDKs directly.
- Delete or weaken the smoke tests we shipped.

D. Common Pitfalls

The headline list (long-form version: `docs/COMMON_PITFALLS.md`):

- **Naked except** – `except Exception: pass` hides bugs. Always catch a specific class, log, re-raise.
- **Unbounded concurrency** – `asyncio.gather` over 200 tasks will get you 429'd by every provider. Use `asyncio.Semaphore`.
- **Hard-coded API keys** – this is a –10 pt automatic deduction. Use `.env`.
- **print()-based diagnostics** – use logging. Configure via env.
- **Tests that hit the network** – mock with `pytest-httpx` or `respx`.
- **Dockerfile that doesn't actually build** – run `docker build` on every PR.
- **All commits from one teammate** – pair on big PRs; share the work; commit independently.

- **Modifying ai/** – –5 pts, also breaks the contract. File an issue instead.
 - **TODOs left in shipped code** – delete them or convert to GitHub issues.
-

Good luck. Build something you would actually ship.